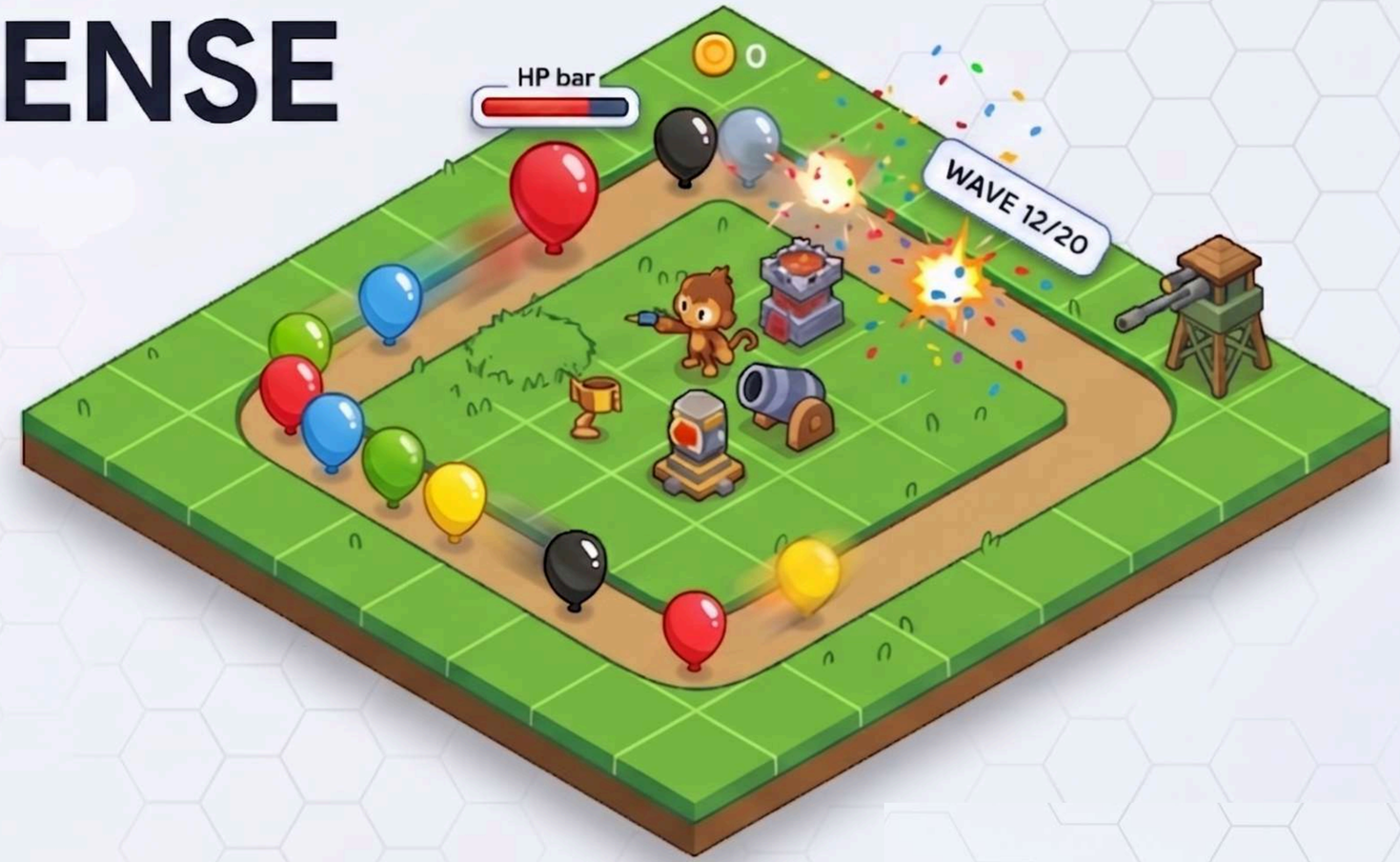
BALLON
TOWER DEFENSE



Sabine Boukais
Son Tung DO
Ines Agueniou
Yanni Moutchou

02 / Introduction

POO & Principe Ouvert-Fermé

L'objectif réel

Objectif pédagogique

Le jeu était un prétexte — l'objectif réel : apprendre à concevoir proprement en POO, avec des classes bien séparées et des responsabilités claires.

Le principe OCP

Open-Closed Principle

> *“Ouvert à l'extension — Fermé à la modification”*

`BombTower` extends `Tower` ✓

Restructuration

Avant ❌
Classe Game monolithique — tout dans un seul endroit

Après ✅
`Game` / `RoundManager` / `GameSetup` / `Board` / `Action`

03 / Avancement

Vue d’ensemble de l’architecture

Organisation par responsabilités — pas par fonctionnalités

`Livrable6 → new GameLoop(new GameSetup(...)).start()`

↓

📦 board

Gestion du plateau, des cellules et des chemins

`Board` · `DefinedBoard` · `FreeBoard` · `Cell` · `Position` · `Direction`

📦 entity

Entités du jeu — ballons, tours, projectiles

`Balloon` · `BalloonType` · `Tower` · `Evolution` `Projectil` · `BombTower` · `IceTower` · `...+4`

📦 choose · action

Interactions joueur & bot — menus et actions

`ListChooser` · `HumanListChooser` · `AutoListChooser` `Action` · `ActionBuilder` · `BuyTowerAction` · `Upgrade` · `Sell`

📦 game · round · ui · clock

Orchestration, affichage et gestion du temps

`GameSetup` · `GameLoop` · `Game` · `RoundManager` `Player` · `Shop` · `GameRenderer` · `BoardRenderer` · `Clock`

💡 `GameContext` *évite de passer tout* `Game` *partout —* `ClockListener` *découple le temps du reste (pattern Observer)*

Choix de modélisation

Gestion du temps & Sélection générique

🎈 Mouvement continu des ballons

Mécanisme

Positions en double + progression interne. Ciblage précis des tours et mouvement fluide sur le chemin.

`Position` · `progression` · `chemin`

Déclic important

Héritage seul → Sert à organiser des types

Command Pattern → Sert à organiser des actions

Le déclic *C'est cette distinction qui fait la vraie différence en conception.*

🎯 ListChooser<T>

Signature

```
interface ListChooser<T> {
   T choose(List<T> options);
}
```

Deux usages distincts

Usages	Humain	Bot
Choisir une action	`HumanListChooser<Action>`	`AutolistChooser`
Choisir une position	`HumanListChooser<Position>`	`Aléatoire`

💡 ***HumanListChooser*** *est devenu générique : il affiche n'importe quel objet via* `toString()` *— un seul composant pour tous les menus.*

État d'avancement

Ce qui fonctionne — détail des fonctionnalités implémentées

✅ Fonctionnel — Ce que le jeu sait faire

Deux types de plateaux

DefinedBoard propose un chemin prédéfini et fixe. FreeBoard permet un placement libre des tours sur n'importe qualle case hors chemin.

`DefinedBoard` · `FreeBoard` · `Board`

Achat / Vente / Évolution de tours

Le joueur gère un budget : acheter de nouvelles tours, vendre celles existantes, ou les faire évoluer vers des versions plus puissantes via Shop.

`Shop` · `BuyTowerAction` · `Upgrade` · `Sell`

Ballons - Déplacement des ballons par rounds

Les ballons progressent case par case le long du chemin. Chaque round fait avancer une nouvelle vague.

`Balloon` · `BalloonType` · `RoundManager`

Mode humain & mode automatique

Le joueur humain interagit via les menus console. BotStrategy gère automatiquement le placement initial et les évolutions selon le numéro de manche.

`BotStrategy` · `HumanListChooser` · `AutoListChooser`

7 types de tours et projectiles

Chaque tour a sa portée, cadence et dégâts propres. Les projectiles sont gérés independamment — BombTower, SniperTower et 5 autres variantes.

`Tower` · `Projectil` · `BombTower` · `SniperTower`

Affichage console centralisé

GameRenderer centralise tout l'affichage : bannière, état des manches, résultats, victoire et game over — BoardRenderer affiche le plateau à chaque tick.

`GameRenderer` · `BoardRenderer` · `GameContext`

Extensibilité du modèle

Étendre sans tout réécrire — grâce aux abstractions

🗺️ Nouveau plateau

Classe abstraite

`Board`

Existant

DefinedBoard · FreeBoard

Extension possible

NouveauBoard

`BoardRenderer` *affiche n'importe quel plateau sans modification*

🗼 Nouvelle tour

Classe abstraite

`Tower`

Existant

BombTower · SniperTower · ...+5

Extension possible

IceTower · HealTower

Ajouter dans `Shop` *— héritage suffit pour la logique de tir*

⚡ Nouvelle action

Classe abstraite

`Action`

Existant

BuyTowerAction · Upgrade · Sell

Extension possible

MoveAction · TrapAction

Ajouter dans `ActionBuilder` *— aucune modification du cœur*

⚠ Limite actuelle Ajouter une tour demande encore d'enregistrer dans `Shop` et parfois d'adapter les projectiles ou les évolutions — pas encore totalement fermé à la modification partout. `Shop` · `Projectil` · `Evolution`

Organisation du travail



Conception collective

UML global réalisé ensemble pour aligner les choix architecturaux avant de coder.

Développement partagé

Code principal écrit en binômes ou en groupe selon les tâches — pas de silo entre les membres.

Livrables tournants

README, tests, documentation et PlantUML répartis de façon rotative entre les membres pour une montée en compétence équilibrée.

`README` · `PlantUML` · `Tests`

Difficultés rencontrées

Deux vrais obstacles — et comment on les a résolus

01 — Où mettre quoi ?

Problème

La création des ballons, la gestion des évolutions — est-ce le rôle du Round, du RoundManager, de Game, ou du Livrable ? On mettait tout dans le Livrable parce que c'est là qu'on testait. Résultat : un Livrable de 200 lignes qui faisait tout le travail à la place de Game.

`Livrable` · `Game` · `RoundManager` · `Round`

Solution

On a restructuré en posant une règle simple : tout ce qui concerne le déroulement du jeu appartient à Game. Le Livrable ne fait plus que configurer et lancer.

Game → logique du jeu | Livrable → configure & lance

`GameSetup` · `Game` · `Livrable`

02 — La Clock

Problème

On avait d'abord un simple compteur qui s'incrémente à chaque tick dans une boucle — mais cette boucle bloque tout, et le rythme dépend de la vitesse du processeur. On ne savait pas comment intégrer la Clock dans Game sans tout coupler.

`Clock` · `GameLoop` · `tick()` ⚠ Boucle bloquante ⚠ Couplage fort

Solution

La Clock tourne dans son propre thread et appelle **onTick()** sur Game à intervalles réguliers via **Thread.sleep()** — Game fait le reste. Résultat : un vrai rythme temporel, et Game et Clock totalement indépendants l'un de l'autre.

`Clock` · `onTick()` · `Thread.sleep()` · `ClockListener`

✅ Thread séparé ✅ Couplage supprimé

Bilan & Améliorations

Bilan positif
Un projet qui a bien progressé

Au-delà des fonctionnalités, on a cherché à rendre le projet plus clair, plus stable et plus solide. L'organisation du travail, la structure du code et le déroulement général ont tous été améliorés au fil des livrables.

Résultat final satisfaisant

Optimisation du code

À améliorer

Certaines classes peuvent encore être optimisées pour être plus légères, plus lisibles et plus faciles à maintenir.

Affichage & joueur automatique

À améliorer

L''affichage peut être amélioré pour être plus clair. Le déroulement du joueur automatique est encore un peu trop long — il gagnerait à être accéléré et mieux structuré.

`BotStrategy` · `GameRenderer`

Tests unitaires

A ajouter

Ajouter des tests unitaires qui couvrent la totalité des classes du projet pour garantir un résultat plus fiable et robuste.